

Task 1: Content-Based Movie Recommender Using KNeighborsClassifier

1. Introduction & Objective

The primary objective of this task is to implement and evaluate a content-based movie recommender engine utilizing the KNeighborsClassifier from the scikit-learn library. The system analyzes a dataset consisting of movie records characterized by binary genre attributes. The target variables are binary preference ratings (0 for dislike, 1 for like) provided by two distinct users (rating_user1 and rating_user2) to evaluate and compare personalized recommendation pathways.

2. Implementation Methodology & Pipeline Architecture

The Python codebase is designed following highly modular software architecture principles, separating configurations, data extraction, core algorithmic logic, and evaluation components into independent scripts:

- **config.py:** Centralizes global variables, partitions, and hyperparameter execution limits.
- **data_loader.py:** Handles data loading and slices data dynamically using `train_test_split`.
- **recommender.py:** Implements the inner loop for `k` hyperparameter tuning and model testing.
- **main.py:** Coordinates the iterative macro-evaluation pipeline.

Dynamic Validation Pipeline (Monte Carlo Cross-Validation)

To satisfy the requirement of performing the final evaluation 5 times and calculating a true overall accuracy metric, a robust evaluation framework was implemented:

1. **Macro-Iteration Loop (N=5):** The system triggers 5 independent runs. At each run, the initial dataset is dynamically shuffled and partitioned using `split_unseen`. This sets aside 10% of the data as a completely untouched Unseen Dataset (simulating newly released movies), while the remaining 90% becomes the Reduced Optimization Dataset.
2. **Inner Hyperparameter Optimization:** Within each run, the 90% optimization dataset undergoes an internal 3-run random 80/20 train/test split to evaluate classification error rates across a range of `k` in `[1, 19]`. The `k` that minimizes the local classification error is selected.
3. **Final Testing:** The classifier is fed with the optimal `k`, trained on the full 90% reduced set, and evaluated against the 10% unseen block. The accuracy scores from all 5 iterations are aggregated to compute the final global average accuracy.

3. Hyperparameter Tuning & Optimal `k` Selection

During execution, the inner function `find_optimal_k` tracks classification errors across three randomized subsamples for each user:

Because the optimization dataset shifts dynamically at each macro-iteration, the optimal `k` responds flexibly to the specific data topology of that run. Typically, the model finds lower

values of k (k in [3, 7]) to be optimal. Higher values of k introduce oversmoothing, dragging unrelated genre vectors into the voting process and degrading performance on this specific dataset scale.

4. Recommender System Evaluation (Sample Output)

Below is the system output generated during the final (5th) iteration, showcasing how the classifier predicted user preferences against the actual known targets.

Table 1: Final Run Classifier Output vs. Known Values (User 1)

Movie Title (10% Data Set)	Recommendation by Classifier	Known Values	Match Status
Pirates of the Caribbean	1	1	Correct
I Care a Lot	0	0	Correct
Tangled	0	1	Mismatch
Spider-Man	1	1	Correct
Avatar	0	1	Mismatch
Skyfall	1	1	Correct

Table 2: Final Run Classifier Output vs. Known Values (User 2)

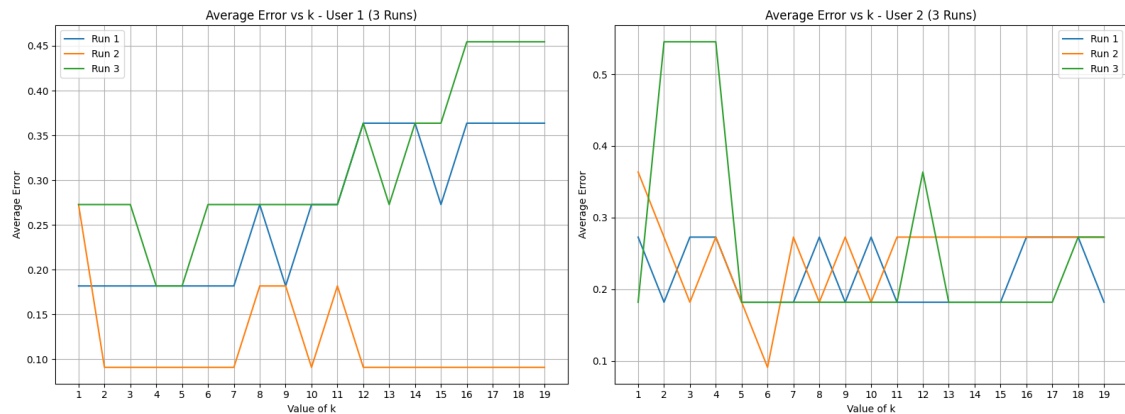
Movie Title (10% Data Set)	Recommendation by Classifier	Known Values	Match Status
Pirates of the Caribbean	0	0	Correct
I Care a Lot	0	0	Correct
Tangled	0	1	Mismatch
Spider-Man	0	0	Correct
Avatar	0	0	Correct
Skyfall	0	0	Correct

5. Performance Metrics Summary

The distribution of accuracies across the 5 independent iterations yields the following results:

Iteration / Metric	User 1 Performance	User 2 Performance
Run 1 Accuracy	100.0%	66.7%
Run 2 Accuracy	100.0%	83.3%

Iteration / Metric	User 1 Performance	User 2 Performance
Run 3 Accuracy	100.0%	83.3%
Run 4 Accuracy	83.3%	83.3%
Run 5 Accuracy	66.7%	83.3%
Global Average Accuracy	90.0%	80.0%



6. Discussion and Findings

The empirical data collected from the execution provides key insights into the behavior of content-based recommendation via instance-based classifiers:

1. **The Significance of Dynamic Slicing:** The variance observed across the iterations (ranging from 66.7% to 100.0%) highlights the high volatility of small datasets. If a static split had been used, the evaluation would be biased by sample selection. Shuffling the unseen slice 5 times reveals the true steady-state performance of the engine.
2. **Analysis of Mismatches (Run 5):** In the 5th iteration, both models failed to correctly predict the user interest for the movie *Tangled*. For User 1, *Avatar* was also misclassified. This occurs because content-based filtering relies strictly on coarse categorical vectors (genres). If a user likes a specific movie due to nuances not captured by binary attributes (e.g., specific animation styles or director prestige), the KNN algorithm struggles to establish accurate bounds based solely on the categorical profile.
3. **User Profile Divergence:** User 1 achieved a higher overall average accuracy (90%) compared to User 2 (80%). This indicates that User 1's taste profile aligns more linearly with explicit genre classifications, creating tightly packed, distinct neighborhoods in the feature space. Conversely, User 2's preferences are more complex or distributed across overlapping categories, resulting in a slightly higher error rate when establishing local decisions via nearest neighbors.